



Pavel Degterev

Exploration of XAI tools

In relation to the FoundedPBI

Metropolia Engineering School

Exchange project in Heig-VD

15.07.26

Abstract or Executive summary

This report documents the exploration of Explainable Artificial Intelligence (XAI) tools applied to the FoundedPBI project - a modular ensemble of genomic foundation models designed to predict Phage-Bacterium Interaction (PBI) from raw DNA sequences. As antimicrobial resistance poses a growing global health threat, developing interpretable tools for phage-host matching is essential. This work focuses on model-importance analysis to understand how individual foundation models (Nucleotide Transformer v2, DNABERT-2, and MegaDNA) contribute to the ensemble's predictions.

We implemented and evaluated multiple perturbation-based methods using the SHAP (SHapley Additive exPlanations) library, including mean ablation (Nb1) and hierarchical partitioning (Nb2). Various baseline strategies - ranging from synthetic means/medians to biologically realistic search-by-ID approaches - were tested to define the reference states for feature attribution. Our findings indicate that while attribution is highly sensitive to methodological choices, hierarchical SHAP provides a more robust and trustworthy diagnostic framework than simple masking. We provide a practical guide for future users, including pipeline integration for automated artifact saving and model diagnostic workflows.

Contents

Glossary

1	Introduction	1
1.1	Report Structure	1
1.2	Biological and Engineering Context	1
1.3	XAI and Shapley Values	2
2	Model Importance	3
2.1	Taxonomy of the methods	4
2.1.1	Algorithmic approaches	4
2.1.2	Implementation of chosen algorithms	5
	Mean ablation (Nb1)	5
	Mean ablation on one organism (Nb1).	6
	Hierarchy SHAP (Nb2)	6
	Clear per-base perturbation (Nb3)	7
2.1.3	Data handling (Baseline Options)	7
	Mean & Median (Nb1 & 2)	8
	Search By Id (Nb1 & 2)	8
	Zeros (Nb1)	8
	Random sampling (Nb2)	9
	Decoupled (Nb2)	9
2.2	Implementation	9
2.2.1	Intended methodology	10
2.2.2	Underlying problems	14
2.3	Usage of the Notebook (friendly manual)	15
2.3.1	Main Configuration Variables	16
2.3.2	Pipeline Overview	16
2.3.3	Parser configuration	17
2.3.4	Project-Side Modifications	18
2.3.5	Possible modification and issues	19
3	Conclusions and upcoming work	20
	Annex A: Additional Resources and Observations	22
	A.1 MegaDNA and Domain Specialization	22

A.2 DNABERT-2 and Cross-Genomic Tasks	22
A.3 Attention Layer-wise Relevance Propagation (AttnLRP)	23
Annex B: PCA Components attribution	24
B.1 Overview and Initial Hypothesis	24
B.2 Observations	24
B.3 Intermediate Results	25

Glossary

bp	base pair – nucleotide pair in the DNA
DNA	Genetic code necessary for an organism to develop, survive, and reproduce.
PCA	Principal Component Analysis
PFI	Permutation Feature Importance
PBI	Phage Bacterium interaction
SHAP	SHapley Additive exPlanations
VRAM	Video Random Access Memory – dedicated memory on a graphics card used to store model parameters, activations, and data during GPU computations; it limits how large models and batches can be loaded at once.
XAI	Explainable Artificial Intelligence

1 Introduction

This report is a companion document to the two (and half) notebooks developed during the exchange project at HEIG-VD, dedicated to discovery of explanation tools available for machine learning. It describes the methods implemented, provides rationale for design choices, and offers a practical guide for future users.

1.1 Report Structure

Section 1 provides important for this project context. **Section 2** describes the model-importance analysis, covering algorithmic methods, baseline strategies, implementation findings, and the notebook usage guide. **Section 3** concludes with practical recommendations and future directions. **Annex A** presents a preliminary exploration of PCA component attributions and outlines ideas for future work in that direction.

1.2 Biological and Engineering Context

Prediction of Phage-Bacterium Interaction (PBI) is an alternative to antibiotics against antimicrobial-resistant bacteria (superbugs). Such bacteria caused approximately 5 million deaths worldwide in 2019 and are projected to cause up to 40 million annual fatalities by 2050. This makes automated, interpretable phage–host matching tools a meaningful public-health contribution.

The CI4CB laboratory at HEIG-VD working in this direction for last decade by using state of the art computational technics. Most recent culmination was in a modular ensemble of pretrained genomic models called FoundedPBI, that predicts PBI from raw DNA sequences alone. It employs three genomic *foundation models* — large transformer-based neural networks pretrained on large DNA sequence databases — as parallel feature extractors:

- **Nucleotide Transformer v2 (NT2)**: an encoder-only transformer trained on ~3,200 human and diverse prokaryotic genomes (single-celled

organisms without a nucleus, like bacteria and archaea), with a context window of 12 K bp and 250 M parameters.

- **DNABERT-2**: a BERT-style model using Byte Pair Encoding (BPE) tokenisation and ALiBi positional encoding, trained on all domains of life except viruses, with a practical context limit of ~16K bp.
- **MegaDNA**: a GPT-style decoder-only transformer trained *exclusively* on 100,000 bacteriophage genomes, with a maximum context of 96K bp.

Each model produces a numerical vector (an *embedding*) that encodes the properties of the input sequence (DNA) as learned during pretraining. For a given phage–bacterium pair, the FoundedPBI backbone generates six embeddings — one per model per organism — that are concatenated into a *meta-embedding* of 18,824 dimensions (amount components in the vector), compressed by PCA to 500 components, and passed to a neural classifier.

1.3 XAI and Shapley Values

Explainable AI (XAI) is the field concerned with making the decisions made by machine learning models interpretable. One of the central concepts is *feature attribution*: assigning a numerical importance score to each input feature to explain a specific prediction.

One of the most straightforward XAI applications in FoundedPBI is block-level attribution across ensemble foundation models. In this setting, each model can be treated as an individual contributor to the final prediction, allowing the analysis of how much predictive information each foundation model provides. Popular tool for this type of task is Shapley-value-based attribution, that originates in cooperative game theory (Shapley, 1953). It provides a principled attribution that fairly distributes the prediction "credit" among features based on their marginal contributions across all possible feature coalitions. The core process of evaluating every possible combination will be further referred to as perturbation.

All explanations usually fall into one of two categories: **local or global**. A *local explanation* explains a single prediction. In context of our project, five phage-bacterium pairs were chosen to match with earlier Federico García's CNN-based XAI study. A *global explanation* aggregates local attributions across many pairs (typically mean absolute Shapley value per block) to describe overall model behavior patterns, such as systematic differences between true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN).

2 Model Importance

During the initial project familiarization and work assessment phase, model importance analysis was selected as the primary exploration objective due to its favourable effort-to-usefulness ratio. Therefore, core goal is to determine which of the three models: NT2, MegaDNA, DNABERT-2 - contribute most to each prediction and does contribution changes between phage and bacteria. In this framing it is possible to determine three concrete use cases:

- **Biological profiling.** When the biological basis of a given PBI pair is known, the models receiving the highest importance may be inferred to encode the corresponding biological information, an interpretation that can be strengthened by comparing importance patterns across pairs with different biological characteristics.
- **Architectural tracking.** From the engineering perspective, attribution analysis helps verify whether finetuning or architectural modifications, such as replacing a one of the foundation models, produce the intended behavioural changes.
- **Error analysis.** Comparing model contributions across true positives, true negatives, false positives, and false negatives also provides a useful framework for identifying systematic prediction errors.

2.1 Taxonomy of the methods

Multiple perturbation methods were implemented to allow testing across different experimental scenarios within a unified, self-contained pipeline. All methods rely on the same underlying principle: the FoundedPBI meta-embedding is constructed from two groups of three foundation models, one for phages and one for bacteria. Individual importance can be evaluated for each one of these six blocks and traced back to the model produced it.

The methods differ in three main aspects: (1) the dimensionality of the perturbation space, ranging from six-dimensional block perturbations to perturbations across the full embedding space of ~18k dimensions; (2) the baseline strategy used to represent an “absent feature” state; and (3) computational cost. Using multiple methods also enables cross-validation, as consistent attribution patterns across methods provide stronger evidence than conclusions derived from a single approach.

2.1.1 Algorithmic approaches

All methods were constructed around an open-source Python library called **SHAP** (**SH**apley **A**dditive **eX**Planations). It provides efficient approximations of Shapley values for a variety of model types (Lundberg & Lee, 2017; [SHAP documentation](#)). Two explainers from this library are used in the present work:

- **KernelExplainer**: A model-agnostic SHAP approximator. It treats the model as a black box and fits a locally weighted linear regression over perturbed feature combinations to estimate Shapley values. Computational cost scales quadratically with the number of features being explained ([SHAP KernelExplainer docs](#)). Implemented in Nb1 & 3.
- **PartitionExplainer**: An explainer that respects a user-supplied *coalition tree* (hierarchical grouping of features) and approximates **Owen values** — a generalisation of Shapley values for games with a pre-defined partition of players into groups (Owen, 1977). Shapley values are computed across

groups first, then within each coalition based on the provided tree structure ([SHAP PartitionExplainer docs](#)). Implemented in Nb2.

2.1.2 Implementation of chosen algorithms

Mean ablation (Nb1)

The original intention was to apply Shapley values to a small number of semantically meaningful “features” (the six embedding blocks) rather than to all 18,824 dimensions. Since SHAP does not offer a built-in “6-feature mask over a large embedding” mode, this idea was implemented explicitly via a *block mask*:

- A 6-dimensional binary vector controls which embedding blocks are active.
- A value of 1 means the original block embedding is used; 0 means it is replaced by a chosen baseline slice.
- KernelExplainer treats this mask as the input feature vector and enumerates all $2^6 = 64$ combinations to compute Shapley values for each block.

This masking strategy makes the method extremely fast, but it also deviates from the “canonical” SHAP setup where feature presence/absence is defined by randomly withdrawing alternative examples from a *background dataset*. Therefore, term **baseline** is deliberately used instead of “absence”. How baselines are constructed and interpreted is discussed in Section 2.1.3. In summary, the baseline should be read as a reference state, not as a neutral or biologically empty state. In practice, this means:

- The method is tightly coupled to the chosen baseline and is not guaranteed to be representative of the full dataset.
- It behaves more like a structured block-level ablation experiment than a textbook SHAP analysis, even though SHAP’s machinery is used to aggregate contributions.

Despite these caveats, this mean-ablation implementation captures the core idea of block-level perturbation: treating each model–organism block as a switchable feature and quantifying its influence on the prediction.

Mean ablation on one organism (Nb1).

This experimental variant was developed to eliminate controversial combinations in which one organism is evaluated against an absent counterpart, potentially generating unrealistic inputs and classifier hallucinations. To address this, the embedding blocks of one organism are kept fixed at their original values, while only the three blocks corresponding to the opposite organism are varied. This results in a 3-dimensional Shapley decomposition on any given pair. Because it is inherently pair-specific and asymmetric, this method was used for targeted local analyses rather than for global summaries.

Hierarchy SHAP (Nb2)

To address the limitations of the masked, baseline-heavy Nb1 approach, Nb2 uses PartitionExplainer directly on the full 18,824-dimensional embedding:

- A coalition tree groups all bacterial embedding dimensions into one coalition and all phage embedding dimensions into another.

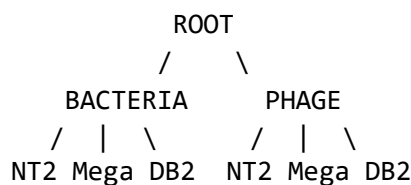


Figure 1. Coalition hierarchy used by Hierarchy SHAP (Nb2).

- Within each coalition, features are treated according to the tree structure, resulting in **Owen-style values** that respect the organism-level grouping.
- The total contribution of each coalition is then divided among its blocks, yielding block-level attributions that explicitly account for dependencies within each organism’s embeddings.

Unlike Nb1, Nb2 uses a *background dataset* rather than a single vector as reference. This provides a more natural reference distribution for Shapley estimation and reduces potential risks associated with a single baseline.

Clear per-base perturbation (Nb3)

This method was considered as a more advanced extension of the first method, enabling “by the book” perturbation at the individual embedding level rather than simple masking. Main advantage is providing SHAP access to a broader background dataset, reducing reliance on a single baseline choice. However, the approach proved computationally prohibitive, with embeddings exceeding 18k dimensions. Therefore, this method was not adopted in the final workflow, but a draft implementation remains available for future exploration as Nb3.

2.1.3 Data handling (Baseline Options)

The choice of baseline or background set is one of the most sensitive design choices in this project, because it defines the reference state from which all SHAP contributions are measured. In principle, different baselines encode different questions.

In this work, all operation were performed on the **test split** extracted during initial model training. An alternative would be to build backgrounds from the **training split**, which would compare new inputs against familiar for the model data. In FoundedPBI, however, the same species appear in both train and test sets, while the **pairings** differ: test pairs are new combinations of familiar organisms. This makes certain baselines being similar independently from their origin. “Mean & Median” discussed further would be one possible example of that. One more possible improvement regarding splitting, would be to retrain the model so that all pairs used in Federico García’s CNN-based XAI study sit in the test set, enabling more systematic cross-comparison of explanations between the two architectures.

This being said, several concrete baseline options were implemented:

Mean & Median (Nb1 & 2)

Here, the baseline is a single vector where each embedding dimension is set to the mean or median across a chosen dataset (for example, all test pairs). These baselines are easy to construct and often produce informative, non-flat importance profiles for both positive and negative predictions.

However, they implicitly assume that “averaging” over embeddings is meaningful. Given that DNA-derived embeddings represent highly diverse biological sequences in a potentially non-linear space, there is no guarantee that a mean or median embedding corresponds to any biologically realistic organism or interaction. Therefore, baseline constructed from real organisms or interactions selected by domain experts remains preferable. Till then, this method might serve as an alternative while expert-defined reference organisms are unavailable.

Search By Id (Nb1 & 2)

To address the interpretability issues of purely statistical baselines, a **Search-by-ID** option was added. Instead of using a mean or median vector, the baseline is a **specific phage–bacterium pair** chosen by the analyst. This provides a biologically realistic reference point for attribution analysis. Furthermore, it enables targeted comparisons between pairs. When two pairs differ in a specific, biologically understood way, differences in block importance may indicate which foundation model is most sensitive to that concept.

Zeros (Nb1)

In the **zero baseline**, all dimensions of a masked embedding block are set to zero. This effectively removes all information from that block and plays a role similar to “turning off” a model component. This produced interpretable behavior: when all blocks from one organism were zeroed, the model prediction collapsed, indicating that the ensemble genuinely relied on information from both organisms.

However, zero baseline is not suitable for explaining negative predictions. In such cases, replacing an already non-contributing embedding with zeros often

produced little or no change in the prediction, limiting the interpretability of the resulting attributions.

Random sampling (Nb2)

For PartitionExplainer, a different approach is implemented. Instead of single baseline vector, `build_background` function yields a dataset with an equal number of interacting and non-interacting pairs (50/50). Class-balanced background provides a more neutral reference distribution than a raw mean baseline and equally suitable for both types of interaction. This option is available only in Nb2 and is likely the most appropriate “baseline” for that notebook.

Decoupled (Nb2)

The decoupled background removes pair-frequency bias by reducing the influence of organisms that are overrepresented in the original distribution. One background row is generated for each unique bacterium and each unique phage, while preserving their original counterpart. The resulting rows are then combined, and duplicate entries removed. This produces explanations that are grounded in species diversity, although it might preserve the original unequal distribution of interactions.

2.2 Implementation

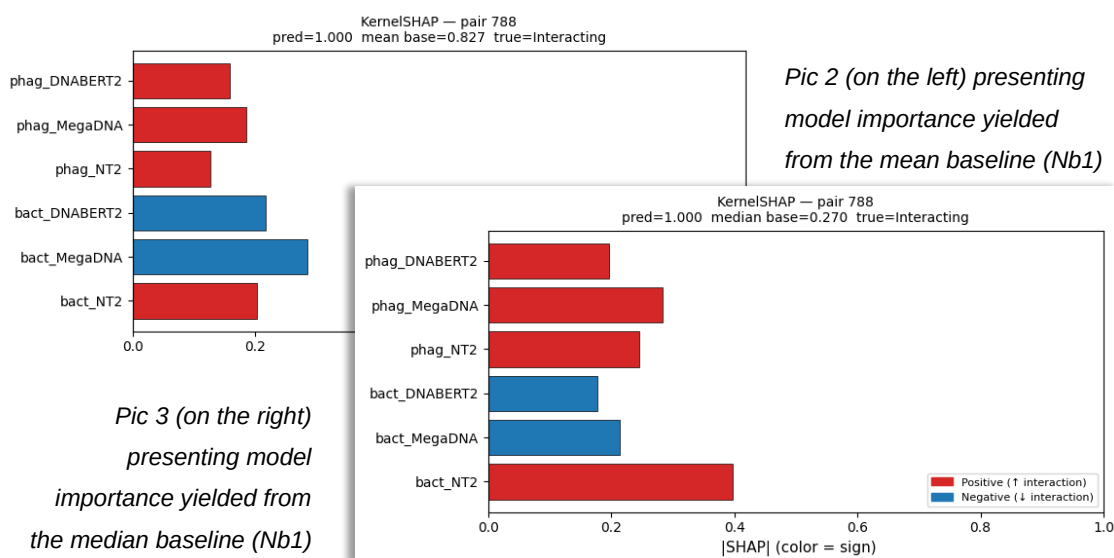
All explanations in this work and in the accompanying notebooks are presented as standardized bar graphs with a consistent structure and maximum length. In this study, positive and negative influences are distinguished by color while being displayed in the same direction, unlike the conventional representation in which positive and negative contributions extend in opposite directions. This design was chosen to facilitate comparison across different scenarios. For the same reason, feature ordering is kept consistent across all visualizations rather than being sorted by attribution magnitude, as is commonly done.

2.2.1 Intended methodology

Based on experiments conducted in both notebooks, a few promising patterns emerged about how the different configurations behave in practice. Together, they suggest a recommended way to use the tools rather than a single “best” setting.

- **Search-by-ID for biologically meaningful comparisons (Nb1 & Nb2).**

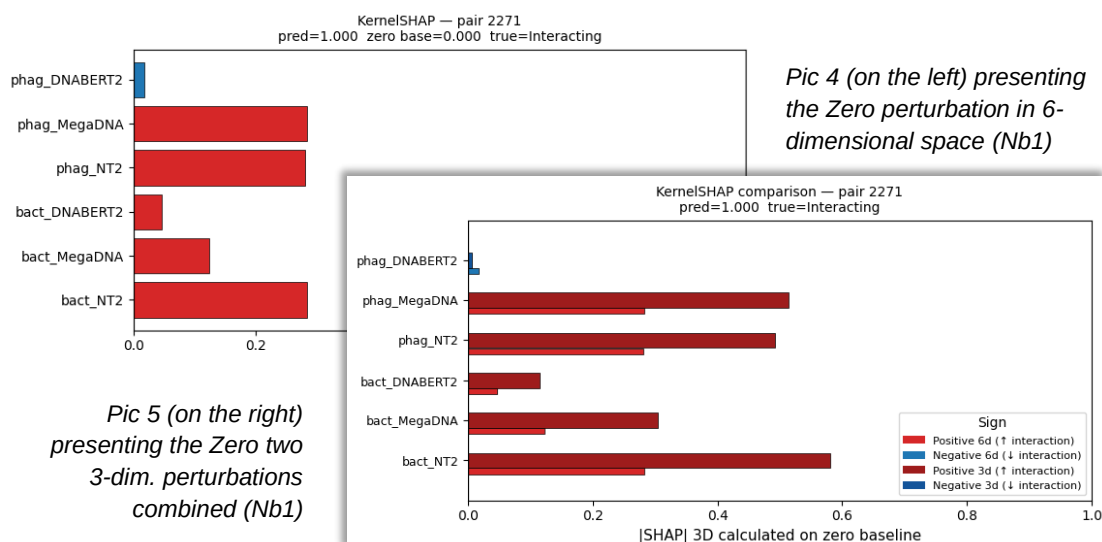
Shap in configuration as it is with the single baseline in the Nb1, produces relative to the baseline importance. It is equally true for Nb2 for inherently same setting with the Mean, Median and Search-by-ID methods. In some cases, the same phage–bacterium pair produced qualitatively opposite explanations depending on the baseline prediction. This best seen in our experiments with mean & median embedding baselines, that had high expected interaction probability for mean (e.g., 0.83, picture 2) or low for median (e.g., 0.27, picture 3).



Based on these findings, we introduced the **Comparison by ID** approach, where a specific phage–bacterium pair is used as the baseline instead of a synthetic mean or median embedding, potentially enabling a deeper analysis of the biological information encoded by individual models.

- **Zero baseline with frozen-side perturbation as a sanity check.**

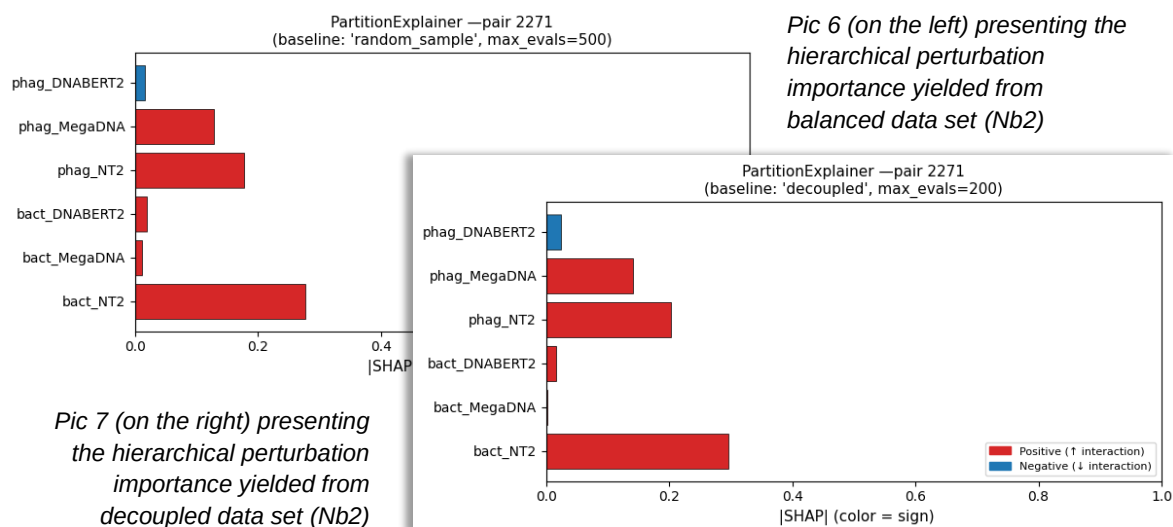
This method was the most stable for analyzing positive interactions among others in Nb1. It was consistent in both the 6-dimensional (all blocks free) and the 3-dimensional setting (one organism frozen). As it will be shown further, this method also passes cross reference check by being relatively consistent with the hierarchy method.



- **Hierarchy SHAP as the most trustworthy method.**

The hierarchical Partition Explainer (Nb2) in opposite for previously described methods provides a complementary view that is less sensitive to a single baseline choice. Furthermore, when run with random sample background (500 pairs) and with the decoupled background (935 pairs), it produced relatively similar block-importance profiles. This cross-reference between backgrounds, including zero-baseline, suggests that hierarchy-based SHAP is the most reliable method in the current toolset, especially because the zero baseline is not suitable for explaining negative predictions. Visual example is based on the same pair as it shown in zero method (Id 2271)

For reproducibility, all hierarchy experiments were run on `RANDOM_SEED = 48` using the saved test dataset in the `output_XAI` folder and switching `BASELINE_MODE` between `random_sample` and `decoupled` in the configuration



- **Global, confusion-matrix explanations as the main diagnostic.**

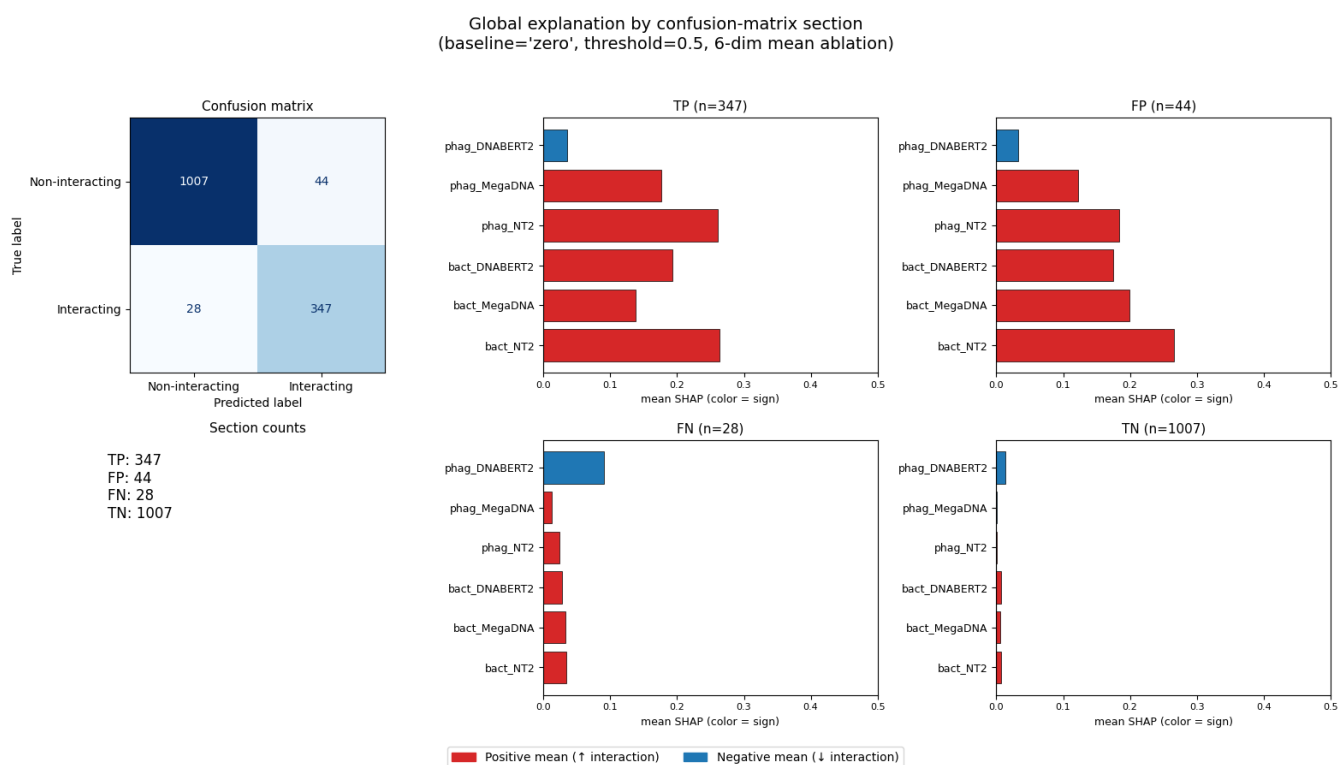
For understanding overall behavior, the most informative view is the global analysis where mean absolute Shapley values are averaged per block separately for true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN). This confusion-matrix breakdown reveals whether certain modes are associated with systematically low (or high) importance in any of those four scenarios, and therefore serves as the main diagnostic output of the notebooks.

Global explanations were generated using the complete test dataset in both notebooks. For the hierarchical approach, a random-sample baseline was used, while the ablation-based approach employed an all-zero baseline. Although the zero baseline negatively affected the interpretability of true-negative explanations, it was considered the most representative choice for the remaining categories and provided a consistent reference for evaluating the contribution of individual embedding blocks.

In this set up, DNABERT2 exhibited unusual behavior, particularly on the phage side of the model. While most foundation models tended to show reduced contribution magnitudes in misclassified samples (FP and FN),

DNABERT2 was comparatively active in these cases in both explanation frameworks. This observation raising the question of whether DNABERT2 was contributing to prediction errors rather than helping the ensemble reach the correct decision.

Although these findings are preliminary, the observed behavior warrants further investigation. Understanding why DNABERT2 has this pattern, and whether it reflects limitations of the model or properties of the training data, it represents a promising direction for future work.

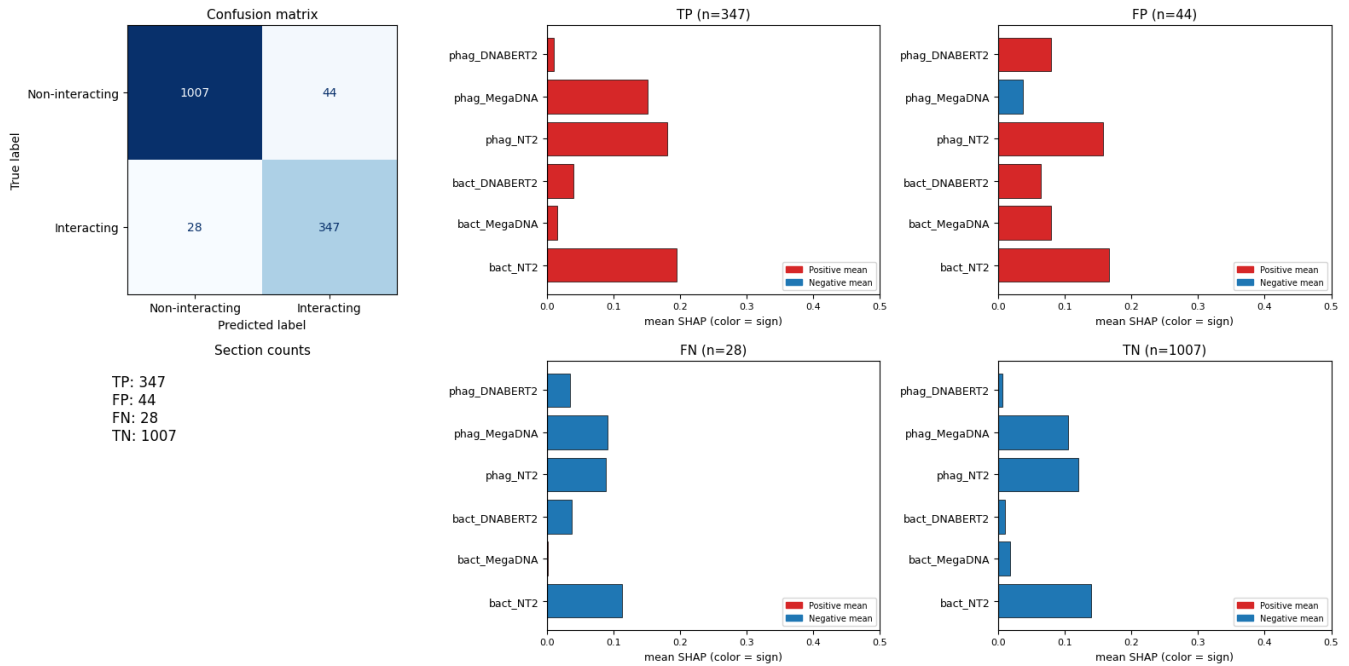


Pic 5, Global explanation with Zero baseline form Nb1

This experiment is also revealed an interesting pattern across both Notebooks. On average in misclassified samples, the foundation models tended to contribute in the same direction, effectively acting as a single voting bloc. In other words, when the ensemble was wrong, each model made same mistakes.

Pic 6, Global explanation with random_sample = 500 from Nb2

Global explanation by confusion-matrix section
(baseline='random_sample', threshold=0.5, 6-dim hierarchy-SHAP)



One possible explanation is that the available sequence context does not contain sufficient information to reliably distinguish between interacting and non-interacting pairs. Under such conditions, the individual foundation models may learn similar misrepresentations and therefore make similar errors, limiting the benefits typically expected from an ensemble. While this interpretation remains speculative, it represents an interesting direction for future investigation.

2.2.2 Underlying problems

Despite the structured methodology, a major challenge observed throughout the experiments was the high sensitivity of explanations to methodological choices. An example is shown in Section 2.2.1, where changing the baseline from the mean to the median fundamentally altered the resulting explanations. Similarly, the same baseline often produced substantially different attribution patterns in Nb1 when comparing frozen-side and full-mask perturbations, and in Nb2 across the corresponding strategies (all experimental outputs are available in the project repository `XAI/jupyter/annex`). This variability makes it difficult to establish a clear deterministic relationship between configuration choices and resulting explanations, leaving the interpretation of individual results highly dependent on

the selected settings. In addition, the limitations of the zero baseline remain unresolved, as it does not represent a meaningful negative interaction. Baseline for interpreting true-negative explanations in Nb1 remains unclear.

As a result, the presented notebooks should be viewed primarily as exploratory diagnostic tools rather than sources of a single, definitive explanation of model behavior. The conclusions that can be drawn from the explanations are inherently dependent on the analyst's objectives and the chosen baseline. Beyond the recommendations provided in the previous section, the interpretation of the generated results ultimately remains the responsibility of the user.

2.3 Usage of the Notebook (friendly manual)

Before running any of the explanation notebooks (Nb1–Nb3), the embedding test dataset must be restored from the compact bundle format. The notebook `restore_compress_emb.ipynb` loads the deduplicated file called `compressedEmbData.pt` from the `XAI/bundle` folder, reconstructs the original data frame, and saves it as `test_dataset.pt` expected by the rest of the project. In practice, only the decompression part of this notebook needs to be executed once per environment; the compression part exists mainly as a reference of what has been used to keep the repository size compatible with version-control limits. Usually, all generated data stored in git-excluded output folder.

All notebooks were done with the similar pipeline for ease of navigation and use. However, here is quick recap of methods implemented in those notebooks. **Nb1** (`nb1_kernel_explainer.ipynb`) implements the 6-dimensional and 3-dimensional mean ablation via the SHAP library and masking. **Nb2** (`nb2_two_level_hierarchy.ipynb`) applies Partition Explainer on the full 18,824-dimensional embedding with a hierarchical tree structure that respects the 3+3 block grouping (bacteria blocks vs. phage blocks). A third notebook, **Nb3** (`nb3_independent_explainer.ipynb`), is stored in the same folder as an experimental extension using per-embedding perturbations and is provided for potential exploration. It is not part of the recommended default workflow.

2.3.1 Main Configuration Variables

The following variables are the primary parameters intended for regular experimental use, assuming the project is configured correctly, and no unexpected issues are present:

Variable	Description
BASELINE_MODE	'mean', 'median', 'random_sample', or 'zero'
N_PAIRS_GLOBAL	Number of pairs for global mean SHAP analysis; use 'all' for the full test set
TARGET_PAIRS	Dictionary of specific pair for local (per-pair) explanation
RANDOM_SEED	Reproducibility seed
CONFIG_PATH	Path to the configuration file used during training

2.3.2 Pipeline Overview

Section 0–3: Configuration, defining block layout, loading data and model.

Section 4: Build the baseline embedding vector in the chosen mode.

Section 5: Notebook specific section: Creation of Block masking class for Nb1 & Creation of Linkage tree for Nb2.

Section 6: Creation of explainer.

Section 7: Definition and implementation of local explanation function.

Section 8: Definition and implementation of global explanation function.

Section 9: Run frozen-side analysis – present only for Nb1.

Visualization sections: local explanations followed by global.

Saving section: All output figures are saved to XAI/results/ using the naming convention including id of PBI and strategy used for explanation. *(Potential improvement: integrate saving paths and conventions into initial configuration section)*

2.3.3 Parser configuration

The explanation notebooks rely on a dedicated configuration parser, `config_parser_XAI`, rather than the original `config_parser` used by the main training and inference pipeline. Its purpose is to implement necessary changes without disturbing original project structure. Those changes aiming automate loading of the objects needed for the explanatory notebooks, and to avoid the unnecessary memory overhead that the default parser introduces by preloading all available models.

Reduced model loading. By default, the original parser loads the full set of embedding models on initialization. The XAI notebooks never need these models at runtime — they operate on pre-computed embeddings restored from the bundle — so the model-loading step in `config_parser_XAI` is intentionally commented out. It reduces occupying GPU memory (VRAM) for models that are not used during explanation.

Self-contained bundle paths. To automate explainability pipeline, the resource paths introduced for this part of the project within config parser. The path definitions located under parser initialization function. However, as a part of finalization they are hardcoded to point at the bundled XAI directory instead of an externally generated training output folder. Example of those as follows:

```
self.model_path = os.path.join("XAI/bundle/trained_model.pth")  
self.bacteria_pca_path = os.path.join("XAI/bundle/pca_bacteria.pkl")
```

This means every resource the notebooks need (the trained classifier, the two PCA objects, and the test dataset) lives in the same `XAI/bundle/` folder, keeping the fork cohesive and portable.

Re-integrating into the original pipeline. When running the original `main.py` training pipeline, the original `config_parser` might need to be updated to include the resource-path definitions described above, but pointing at the training output directory rather than the bundle:

```
self.phage_pca_path = os.path.join(self.output_dir, "pca_phage.pkl")
self.test_path = os.path.join(self.output_dir, "test_dataset.pt")
```

These additions can be copied directly from `config_parser_XAI`. With these few changes in place, together with the project-side modifications listed in next chapter - the full flow runs automatically, and the notebooks can consume freshly trained artifacts without manual intervention. With the exception mentioned in 2.3.3 in regard if new models will be introduced.

2.3.4 Project-Side Modifications

The changes below were introduced in the original project to improve end-to-end flow (training → artifact saving → explanation). Changes are small, but some of them critical (such as PCA saving during training pipeline); the approximate line numbers are provided as a navigation aid only and may shift as the codebase evolves.

Change	Location (approx.)	Purpose / notes
PCA auto-save	main.py, lines ~308–330 (config. -bacteria_pca_path, -phage_pca_path)	Saves the phage and bacteria PCA components automatically upon creation via <code>pickle.dump(...)</code> . Not implemented in the original codebase.
Test-set saving	main.py, lines ~932–943 (config.test_path)	Saves the test dataset after the train/test split via <code>torch.save(test, config.test_path)</code> <i>Currently a draft.</i> The split happens without oversampling in mind. So already-oversampled dataset would duplicate instances across train and test and cause data leakage — a more careful approach (e.g. oversampling only after the split) likely deserves attention.
Model saving	main.py, lines ~984–987 (config.model_path)	Saves the trained model to a fixed path so the notebook flow can locate it automatically. <i>The model file name and location was hardcoded in the main.</i>

Once these modifications are present (and the matching path definitions are copied from `config_parser_XAI` into the original `config_parser`, as described in 2.3.3), the training run produces an output folder containing the trained model, run statistics, the test dataset, and the two PCA objects — exactly the artifacts the explanation notebooks expect under Section 3 (Data & Model Loading). *(Note: test-set export deserves exceptional attention to avoid oversampling)*

2.3.5 Possible modification and issues

All explanations in the notebooks are computed on top of the final classifier used in the original FoundedPBI project. The structure of this classifier, as well as the paths to the test dataset and PCA objects, is tightly coupled to the existing project layout and configuration files. Therefore, parent project environment may need to be regenerated according to the instructions in the root folder of the original repository before these notebooks can be used reliably.

Notebooks are designed to be run from the project root directory (`data/your.user/pbi/`). All primary parameters are configured in **Section 1 (Configuration)**; Under standard experimental conditions, no other notebook cells should require modification.

Possible exceptions are listed below:

- **Model implementation changes (highly likely):**

Section 3 – Data & Model Loading assumes that all changes made to the model `config` class have already been implemented and works automatically. If this is not the case, the test dataset must be loaded manually. The same applies to the PCA component, which should be saved during model training and stored in automatically generated during model training output folder. This folder should contain trained model, some stats, test dataset, 2 PCA objects for phage and bacteria.

Any required modifications for this functionality are located at the beginning of the section. *(Possible point of improvement, to make it less depended on the initial codebase)*

- **Changes to the model configuration:**

Section 2 – Block definitions contain the model names and their alignment within the final concatenated embedding space. This section must be updated manually if the configuration file differs from the original setup, for example: new models being integrated or current removed, model arrangement in the configuration file was changed etc. *(Potential improvement: dynamically generate block definitions directly from the configuration file.)*

- **Changes to data handling or baseline methodology:**

Section 4 – Baseline & Dataset manages the reference dataset generation and baseline handling logic. This section might be modified if: new baseline methods are introduced, preprocessing strategies are changed, or functionality is migrated between notebooks.

The implementation currently relies on a long if statement handling all supported variations of the BASELINE_MODE variable. *(Potential improvement: replace the current implementation with an Enum and structured switch-like handling for improved maintainability.)*

3 Conclusions and upcoming work

This project showed that Shapley-based methods can be useful diagnostic tools for model-level analysis within the FoundedPBI pipeline. While they are not a source of final or absolute truth, they provide a valuable framework. During this work, several useful methodologies emerged, providing insights into the contributions of the underlying genomic foundation models and yielding several primary conclusions regarding tool reliability and model diagnostics.

Simple mean ablation is fast and easy to interpret, but its results can change substantially depending on whether a mean, median, or zero reference is used. For that reason, Search-by-ID appears to be the most promising option for biologically meaningful comparisons and an interesting direction for future work. However, in practice, its usefulness is limited by the lack of direct biological expertise needed to define suitable reference cases, while the scale of the current project does not justify allocating sufficient resources to such analyses.

Hierarchy SHAP appears to be the most robust diagnostic tool within the current framework. Compared with simple masking, it operates on a richer background distribution and therefore produces more stable and reliable results. Nevertheless, the final choice of methodology will depend on the goals of the analyst, as there is no definitive answer regarding which approach is universally preferable.

The global error analysis also revealed an important observation: the ensemble often fails in a coordinated manner, with the underlying models making the same types of mistakes simultaneously. This suggests a possible limitation related to insufficient sequence context, preventing the models from fully capturing the biological interaction. In addition, DNABERT-2 exhibited potentially unusual behaviour during misclassifications, further demonstrating that the proposed approach can be useful for identifying anomalies even when their exact biological interpretation remains unresolved.

Overall, this work does not solve the interpretability problem, but it identifies areas where the current system may break down and highlights directions for future improvements. In that sense, this project is less an endpoint than a practical foundation for the upcoming Bachelor's thesis. Future efforts will focus on moving beyond post-hoc explanations toward a more transparent architecture through cross-attention-based interaction modelling and expanded sequence context windows.

Annex A: Additional Resources and Observations

The following resources were identified during the project but were not explored in detail due to time constraints, project scope, or lack of domain expertise. They may provide useful starting points for future investigations.

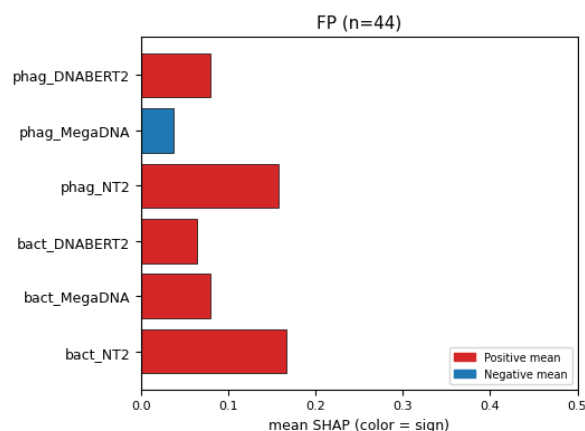
A.1 MegaDNA and Domain Specialization

<https://www.nature.com/articles/s41467-024-53759-4>

This paper discusses the design and training strategy of MegaDNA, which was developed specifically for bacteriophage genomes. One possible interpretation is that MegaDNA may be naturally better suited for phage-related representations than for bacterial representations. This could potentially be related to some of the attribution patterns observed during the global explanation experiments (particularly the false-positive cases).

However, this interpretation remains speculative and was not investigated during the current project. The paper is included here as a potentially relevant reference for future work.

Part of pic 6, Global explanation from Nb2



A.2 DNABERT-2 and Cross-Genomic Tasks

<https://www.nature.com/articles/s41467-025-65823-8>

This paper presents a comparative evaluation of genomic foundation models across a variety of downstream tasks. An interesting observation is that DNABERT-2 does not consistently surpass in cross-genomic tasks.

This may potentially relate to the unusual behaviour observed for DNABERT-2 during the attribution experiments. I am not confident enough in my interpretation to draw any conclusions, but the paper contains several interesting benchmark tables that may be useful for someone investigating this question in greater depth.

A.3 Attention Layer-wise Relevance Propagation (AttnLRP)

<https://github.com/rachtibat/LRP-eXplains-Transformers>

This framework was originally considered for the project but ultimately fell outside the available scope and timeline. Nevertheless, it appears to be a well-established and actively used explainability framework for transformer architectures.

Particularly interesting would be the possibility of generating importance scores directly on DNA sequences and comparing them with the model-level importance scores explored in this report. Such a comparison could provide additional insight into whether model-level importance correlates with sequence-level attention patterns and may represent a natural continuation of the current XAI exploration.

Annex B: PCA Components attribution

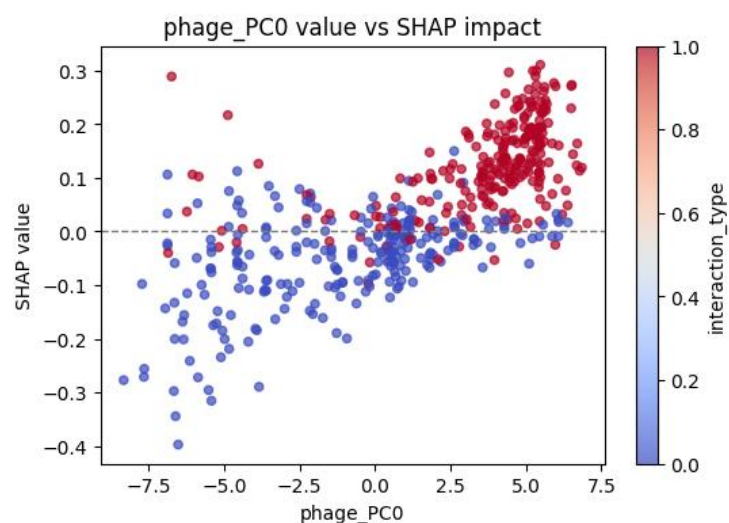
B.1 Overview and Initial Hypothesis

The initial goal of this exploration was to determine if the classifier's predictions were driven by any easily identifiable features captured within Principal Component Analysis (PCA) objects. We hypothesized that if specific PCA components contained sufficient information or represented latent biological features, we might be able to map "successful" clusters on a PCA plot to actionable, human-readable rules. In other words, if certain areas within a PCA object - or specific ranges of a component's values - consistently correlated with positive interactions, we could extract these as "rules" to explain model reasoning and potentially gain insight into the underlying mechanics of Phage-Bacterium Interactions (PBI) by mapping those areas to the models or even DNA sequence.

B.2 Observations

We analyzed the relationship between PCA component values (x-axis) and the SHAP approximation of the model outcome (y-axis) to visualize how variations in these components lean toward positive or negative predictions.

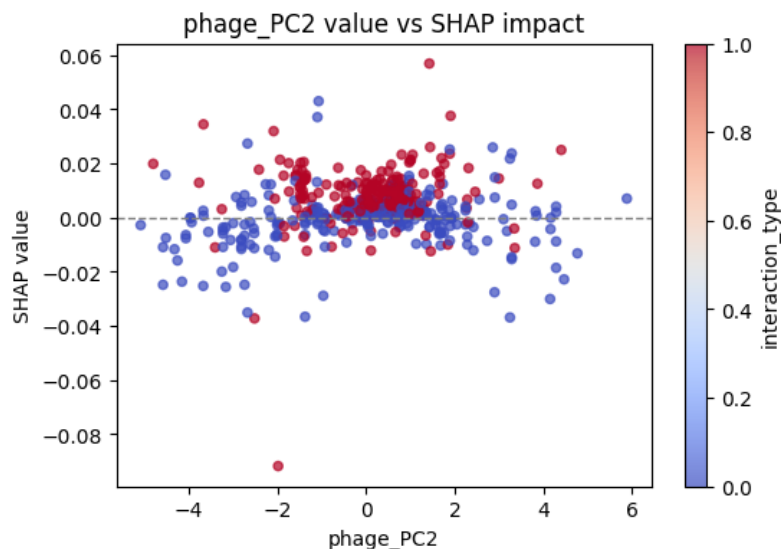
- **Positive Cases (e.g., phage_pc0):** As shown in the figure below, some components exhibit localized clustering where positive interactions appear to aggregate. This suggested a potential for feature extraction based on spatial mapping.



Pic 7, First and therefore the most valuable for the classifier PCA component on the phage side

- **Negative/Inconclusive Cases (e.g., phage_pc2):** Conversely, most other components - as seen in the example below - presented as disorganized, "messy" distributions with no clear grouping dependent on the PCA value.

Pic 8, Third PCA component on the phage side, generated within same test run



B.3 Intermediate Results

After systematic testing, this approach was dismissed in favor of the model importance perturbation methods, which yielded more reliable and interpretable results. Our technical findings were as follows:

Lack of Correlation: While there were indications of patterns within few components, no significant linear correlations were found between PCA components and model outcomes.

Implementation Status: While we developed multiple approaches for recreating and analyzing PCA objects, only the most robust implementation was preserved in the project's GitHub repository.

Conclusion: This section serves as a concept. Database yielded from experiments presented in the `pca_shap_testing.ipynb` contains several anomalous observations; however, these remain undocumented and require further investigation should future research suggest that PCA-based attribution is a viable path for the FoundedPBI project.